

UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional San Rafael
Ingeniería en Sistemas de Información

ELECTIVA: ARDUINO

Trabajo Final Integrador

Torreta Láser con Seguimiento Ultrasónico

*Sistema autónomo de detección, localización y apuntado de
objetos basado en Arduino UNO R3*

INTEGRANTES

Branko Almeida · Federico Sosa · Agustín Giorlando
Lisandro Toledo · Agustín Lara

Docentes: Ing. Alfredo G. Rivamar — Ing. Pablo Moliterno

2026

Resumen

El presente trabajo describe el diseño, la implementación y la validación de una **torreta láser autónoma** capaz de detectar la presencia de objetos en su entorno, localizarlos angular y radialmente mediante un barrido ultrasónico, y apuntar un haz láser hacia el objetivo más cercano. El sistema se desarrolla sobre una placa **Arduino UNO R3** e integra tres sensores (ultrasónico HC-SR04, sensor pasivo infrarrojo PIR HC-SR501 y fotorresistencia LDR) junto a cuatro actuadores (dos servomotores SG90 en configuración pan-tilt, un diodo láser, un buzzer piezoeléctrico y una pantalla LCD 16×2 mediante interfaz I2C). La lógica de control implementa una máquina de estados finitos con una rutina de servicio de interrupción asociada al sensor PIR para el disparo inmediato del modo de búsqueda, cumpliendo así con el requisito de uso de interrupciones para eventos críticos. Se desarrollan los modelos teóricos asociados al sensado por tiempo de vuelo acústico, la compensación de la velocidad del sonido por temperatura, la conversión de coordenadas polares a posiciones angulares de servo y el control por modulación por ancho de pulso. Se discuten asimismo las limitaciones del sensado ultrasónico —en particular la apertura del haz— y las estrategias de filtrado y estimación de centroide implementadas para mitigarlas. El prototipo, validado por simulación en Tinkercad/Wokwi y por implementación física, demuestra una localización angular efectiva con una resolución del orden de pocos grados, suficiente para el apuntado del haz láser dentro del rango operativo del sistema.

Palabras clave: Arduino, sensado ultrasónico, control de servomotores, máquina de estados, interrupciones, fusión de sensores, sistemas embebidos.

Índice

1. Introducción y objetivos
2. Marco teórico
 1. Microcontrolador Arduino UNO R3
 2. Sensores empleados
 3. Actuadores empleados
3. Diseño del sistema
 1. Arquitectura general
 2. Diagrama de bloques
 3. Esquema eléctrico y conexionado
 4. Diseño de la maqueta
4. Desarrollo del software
 1. Máquina de estados
 2. Estructura del sketch y funciones
 3. Uso de interrupciones

4. Algoritmo de detección y seguimiento
5. Filtrado y estimación de centroide
5. Modelos teóricos y ecuaciones
 1. Distancia por tiempo de vuelo acústico
 2. Compensación de temperatura
 3. Coordenadas polares a ángulos de servo
 4. Control por modulación por ancho de pulso
 5. Apertura del haz y precisión angular
6. Resultados y validación
7. Conclusiones
8. Bibliografía

1. Introducción y objetivos

1.1. Contexto y motivación

La integración de microcontroladores de bajo costo, sensores y actuadores ha extendido en la última década el espectro de aplicaciones de la electrónica embebida desde la industria hacia ámbitos educativos, domésticos y de investigación. En este contexto, la plataforma Arduino constituye un estándar de facto para el prototipado rápido por su accesibilidad económica, su comunidad activa y la riqueza de su ecosistema de bibliotecas.

El presente trabajo aborda una problemática típica de los sistemas de vigilancia y de la robótica básica: la *detección y localización de objetos en un entorno bidimensional*. Inspirado en la analogía de un radar, el sistema realiza un barrido angular del espacio mediante un sensor ultrasónico montado sobre un servomotor y, una vez identificado el blanco más cercano, orienta un haz láser hacia el mismo. Si bien el caso de uso ilustrativo es lúdico o de demostración, los principios involucrados —sensado activo, barrido programado, fusión de información de múltiples sensores, control de actuadores en lazo cerrado y respuesta a eventos asíncronos— son representativos de problemas de ingeniería ampliamente difundidos en la automatización industrial, la robótica móvil y los sistemas de seguridad perimetral.

1.2. Objetivo general

Diseñar, programar y prototipar un sistema autónomo de detección y apuntado, basado en Arduino UNO R3, que integre al menos tres sensores y tres actuadores, y que responda a eventos críticos mediante interrupciones de hardware, conforme a los requisitos establecidos en la consigna del Trabajo Final Integrador de la cátedra.

1.3. Objetivos específicos

- Diseñar la arquitectura de hardware del sistema, seleccionando justificadamente cada uno de los componentes en función de sus especificaciones técnicas.
- Desarrollar un sketch organizado en funciones modulares y debidamente comentado, que implemente una máquina de estados finitos para la coordinación del sistema.
- Incorporar al menos una rutina de servicio de interrupción asociada a un evento crítico (detección de presencia).
- Formular los modelos teóricos que sustentan el funcionamiento del sistema, incluyendo el sensado ultrasónico, la conversión de coordenadas y el control de servomotores.
- Implementar un esquema de filtrado y estimación que mitigue la limitada resolución angular del sensor ultrasónico.

- Validar el funcionamiento del sistema mediante simulación en Tinkercad/Wokwi y mediante un prototipo físico sobre una maqueta de bajo costo.

1.4. Alcance

El sistema desarrollado opera en un plano horizontal en un rango angular de 0° a 180° y en distancias comprendidas entre los 2 cm y aproximadamente 200 cm —límites prácticos impuestos por el sensor HC-SR04—. La resolución angular efectiva queda determinada por la apertura del haz ultrasónico, según se discute en el apartado correspondiente. El sistema no contempla, en su versión base, el reconocimiento del tipo de objeto detectado: trata todo eco dentro del umbral configurado como un blanco válido. Se proponen, no obstante, posibles extensiones en las conclusiones.

2. Marco teórico

2.1. Microcontrolador Arduino UNO R3

La placa Arduino UNO R3 está construida en torno al microcontrolador ATmega328P de Microchip Technology, una unidad de 8 bits de la familia AVR con arquitectura RISC. Sus características más relevantes para este proyecto se resumen en la Tabla 2.1.

Parámetro	Valor
Microcontrolador	ATmega328P
Frecuencia de reloj	16 MHz
Memoria de programa (Flash)	32 kB (0,5 kB usados por el bootloader)
SRAM	2 kB
EEPROM	1 kB
Pines E/S digitales	14 (6 con capacidad PWM)
Entradas analógicas	6 (resolución 10 bits, 0–5 V)
Interrupciones externas	2 (pines D2 e D3)
Corriente máxima por pin E/S	20 mA (40 mA absoluto)
Tensión de operación	5 V

Tabla 2.1. Características técnicas relevantes del Arduino UNO R3.

De los parámetros anteriores resultan especialmente significativos para el presente desarrollo: la disponibilidad de seis salidas con modulación por ancho de pulso (PWM), necesaria para el control de los

servomotores; la existencia de dos pines con capacidad de interrupción externa, sobre los cuales se conecta el sensor PIR; y la presencia de un módulo conversor analógico-digital (ADC) de 10 bits, empleado para la lectura de la fotorresistencia.

2.2. Sensores empleados

2.2.1. Sensor ultrasónico HC-SR04

El HC-SR04 es un sensor ultrasónico activo que mide la distancia a un objeto mediante el principio del tiempo de vuelo (*Time of Flight*, ToF) acústico. El módulo dispone de dos transductores piezoeléctricos: uno transmisor, que emite una ráfaga de ocho ciclos a una frecuencia de 40 kHz al recibir un pulso de disparo en el pin TRIG de al menos 10 μ s de duración; y uno receptor, que detecta el eco reflejado por el obstáculo. La salida ECHO se mantiene en alto por un tiempo proporcional al lapso entre la emisión y la recepción del eco, conforme al modelo desarrollado en la sección 5.1.

Parámetro	Valor
Tensión de alimentación	5 V
Corriente de operación	15 mA
Frecuencia ultrasónica	40 kHz
Rango de medición	2 cm – 400 cm
Precisión nominal	± 3 mm
Apertura efectiva del haz	$\approx 15^\circ$ (cono total $\approx 30^\circ$)
Tiempo mínimo entre lecturas	60 ms

Tabla 2.2. Especificaciones del sensor HC-SR04.

La principal limitación del sensor para aplicaciones de localización angular es la apertura del haz: el lóbulo principal del transductor de 40 kHz tiene una directividad limitada, lo que implica que el sensor no resuelve adecuadamente la posición angular de objetos pequeños. Esta limitación se aborda en el apartado 5.5 mediante un esquema de barrido fino con estimación de centroide.

2.2.2. Sensor de movimiento PIR HC-SR501

El HC-SR501 es un sensor pasivo infrarrojo (*Passive Infrared*, PIR) basado en un detector piroeléctrico de doble elemento sensible a la radiación infrarroja del rango 8–14 μ m, característica de los cuerpos a temperatura ambiente. Una lente de Fresnel sobre el sensor segmenta el campo de visión en zonas: el desplazamiento de una fuente de calor entre estas zonas genera una variación diferencial de tensión en el elemento piroeléctrico, que el circuito integrado del módulo (BISS0001) convierte en una señal digital activa en alto. El módulo expone dos potenciómetros para ajustar la sensibilidad y el tiempo de retención de la señal.

En este proyecto, el PIR cumple la función de *disparador del modo de búsqueda*: cuando el sistema se encuentra en estado de espera, la detección de movimiento por parte del PIR genera una interrupción que despierta al sistema y lo lleva al estado de barrido. Este uso es representativo del concepto de *evento crítico* exigido por la consigna, dado que la latencia entre el evento físico (movimiento) y su procesamiento debe minimizarse.

2.2.3. Fotorresistencia (LDR)

La fotorresistencia o resistor dependiente de la luz (*Light Dependent Resistor*, LDR) es un componente cuya resistencia eléctrica varía inversamente con la intensidad luminosa incidente. El elemento sensible está construido típicamente con sulfuro de cadmio (CdS), cuya conductividad aumenta con la fotoexcitación de portadores de carga. La caracterización del componente se ajusta razonablemente al modelo de ley de potencia:

$$R_{\text{LDR}} = R_{10} \cdot (E / 10 \text{ lx})^{-\gamma}$$

donde R_{10} es la resistencia nominal a 10 lux, E es la iluminancia y γ es el coeficiente de sensibilidad espectral (típicamente entre 0,7 y 0,9 para LDRs comerciales).

En esta aplicación el LDR se utiliza como sensor de luz ambiente para dos propósitos: (i) habilitar un *modo nocturno* en el cual el sistema entra en estado de reposo cuando la iluminación supera un umbral, y (ii) servir como entrada adicional para la verificación cruzada del modo operativo. Se conecta en configuración de divisor de tensión con una resistencia fija de 10 k Ω , y se digitaliza mediante el ADC de 10 bits del Arduino.

2.3. Actuadores empleados

2.3.1. Servomotor SG90

El SG90 es un servomotor de modelismo de bajo costo y reducido tamaño (9 g), compuesto por un motor DC de núcleo de hierro, un tren de engranajes plásticos con reducción del orden de 1:300, un potenciómetro acoplado al eje de salida como sensor de posición, y un circuito electrónico de control que cierra el lazo de posición. El servo recibe la consigna de posición angular como una señal de pulso periódica de aproximadamente 20 ms (50 Hz), cuyo ancho determina la posición del eje: 1 ms corresponde a 0°, 1,5 ms a 90° y 2 ms a 180° (la respuesta exacta puede variar levemente entre unidades).

Parámetro	Valor
Tensión de operación	4,8 – 6 V
Torque (stall)	1,8 kg·cm
Velocidad angular	0,1 s / 60° (a 4,8 V)
Rango angular	0° – 180°
Consumo en movimiento	≈ 250 mA
Resolución práctica	≈ 1°

Tabla 2.3. Especificaciones del servomotor SG90.

En el sistema se emplean dos servos en configuración pan-tilt: uno horizontal, responsable del barrido y del posicionamiento azimutal del conjunto sensor-láser, y uno vertical, responsable de la corrección en elevación durante el apuntado fino.

2.3.2. Diodo láser

Se utiliza un módulo láser de diodo semiconductor de baja potencia (típicamente 5 mW, clase IIIa, longitud de onda 650 nm — rojo) alimentado a 5 V. La función del láser es estrictamente indicativa: señalar visualmente la posición del blanco detectado. Por razones de seguridad ocular, el módulo se opera en modo pulsado mediante un pin digital, evitando la exposición prolongada y permitiendo además la indicación parpadeante del modo "objetivo fijado".

2.3.3. Buzzer piezoeléctrico

El buzzer piezoeléctrico convierte una señal eléctrica oscilante en una onda sonora audible por deformación mecánica de un disco piezoeléctrico bajo un campo eléctrico variable. Se emplea un buzzer activo de 5 V, que incluye internamente el oscilador y produce un tono fijo al recibir tensión continua. Su función es señalar acústicamente la detección de un blanco, distinguiendo mediante patrones temporales los estados de *blanco detectado*, *blanco fijado* y *alarma*.

2.3.4. Pantalla LCD 16×2 con módulo I2C

La pantalla de cristal líquido alfanumérica de 16 caracteres por 2 líneas, controlada por el chip Hitachi HD44780, se utiliza para la presentación al operador del estado del sistema, el ángulo de barrido, la distancia al blanco y el modo activo. Para reducir el uso de pines del Arduino, se incorpora un módulo de expansión basado en el PCF8574, que adapta la interfaz paralela del LCD a un bus I2C de dos hilos (SDA y SCL).

3. Diseño del sistema

3.1. Arquitectura general

El sistema se organiza en torno al microcontrolador como elemento central de procesamiento, con tres bloques funcionales conectados al mismo:

- **Bloque de sensado:** compuesto por el sensor ultrasónico HC-SR04, el sensor PIR HC-SR501 y la fotorresistencia LDR.
- **Bloque de actuación:** compuesto por los dos servomotores SG90, el diodo láser y el buzzer.
- **Bloque de presentación:** compuesto por la pantalla LCD 16×2 con interfaz I2C.

3.2. Diagrama de bloques

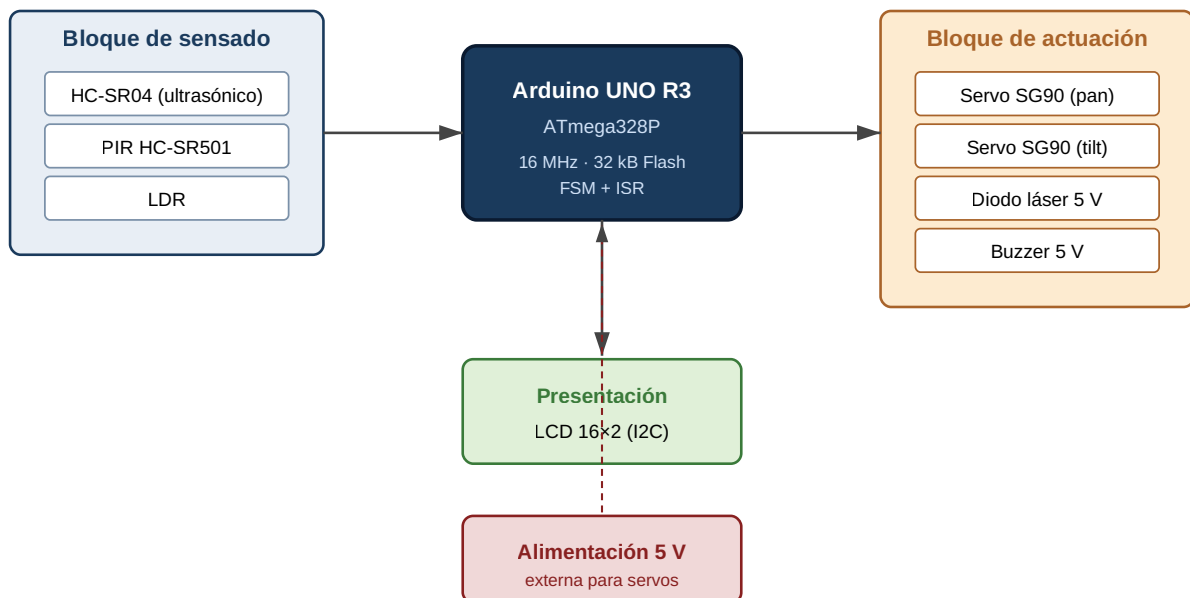


Figura 3.1. Diagrama de bloques del sistema. Las flechas continuas indican flujo de información; la flecha punteada indica alimentación.

3.3. Esquema eléctrico y conexionado

La Tabla 3.1 detalla la asignación de pines del Arduino UNO R3 a cada uno de los componentes del sistema. La elección de pines respeta las siguientes restricciones funcionales: los servomotores se conectan a pines con capacidad PWM; el sensor PIR se ubica obligatoriamente en uno de los pines con capacidad de interrupción externa (D2 o D3); el módulo LCD se conecta al bus I2C en los pines dedicados A4 (SDA) y A5 (SCL); y la fotorresistencia se conecta a una entrada analógica.

Componente	Pin de señal	Tipo	Alimentación
HC-SR04 — TRIG	D7	Salida digital	5 V / GND
HC-SR04 — ECHO	D8	Entrada digital	—
PIR HC-SR501	D2	Entrada digital (INT0)	5 V / GND
LDR (divisor c/ 10 kΩ)	A0	Entrada analógica	5 V / GND
Servo SG90 — pan	D9 (PWM)	Salida PWM	5 V ext. / GND común
Servo SG90 — tilt	D10 (PWM)	Salida PWM	5 V ext. / GND común
Diodo láser	D6	Salida digital	5 V (a través de pin)
Buzzer activo	D5	Salida digital	5 V (a través de pin)
LCD 16×2 — SDA	A4	I2C — SDA	5 V / GND
LCD 16×2 — SCL	A5	I2C — SCL	5 V / GND

Tabla 3.1. Asignación de pines del Arduino UNO R3.

Nota sobre la alimentación: los servomotores SG90 presentan un pico de corriente de arranque considerable ($\approx 600\text{--}800\text{ mA}$ en stall) que excede ampliamente la capacidad del regulador interno del Arduino. Por ello se prevé una alimentación externa de 5 V dedicada a los servos, con la masa unificada a la del Arduino para garantizar la referencia común de la señal PWM. Adicionalmente, se incorpora un capacitor electrolítico de 470 μF en los bornes de alimentación de los servos para mitigar las caídas transitorias de tensión.

3.4. Diseño de la maqueta

La maqueta se concibe con un criterio de mínimo costo y máxima visibilidad demostrativa. La estructura física se compone de:

- Una **base de cartón pluma o MDF de $30 \times 30\text{ cm}$** que aloja el Arduino, la fuente de alimentación y la cableada general.
- Un **soporte pan-tilt** impreso en 3D o construido en cartón pluma, sobre el que se montan los dos servomotores en configuración perpendicular; el servo superior soporta el conjunto solidario HC-SR04 + diodo láser, mecánicamente alineados.
- Un **"campo de juego"** de aproximadamente $60 \times 60\text{ cm}$ en cartón, con marcas semicirculares cada 30° a modo de transportador, sobre el que se colocan los objetos de prueba (bloques de $5\text{--}10\text{ cm}$).
- Un **tablero de información** vertical donde se monta el LCD 16×2 y un indicador LED del estado del sistema.

La elección de cartón pluma y MDF responde a su accesibilidad, bajo costo y facilidad de mecanizado con herramientas manuales, sin comprometer la rigidez requerida para que los servomotores no introduzcan vibraciones espurias sobre la línea visual del láser.

4. Desarrollo del software

4.1. Máquina de estados

La lógica del sistema se modela como una máquina de estados finitos (*Finite State Machine*, FSM) con cinco estados, según se ilustra en la Figura 4.1.

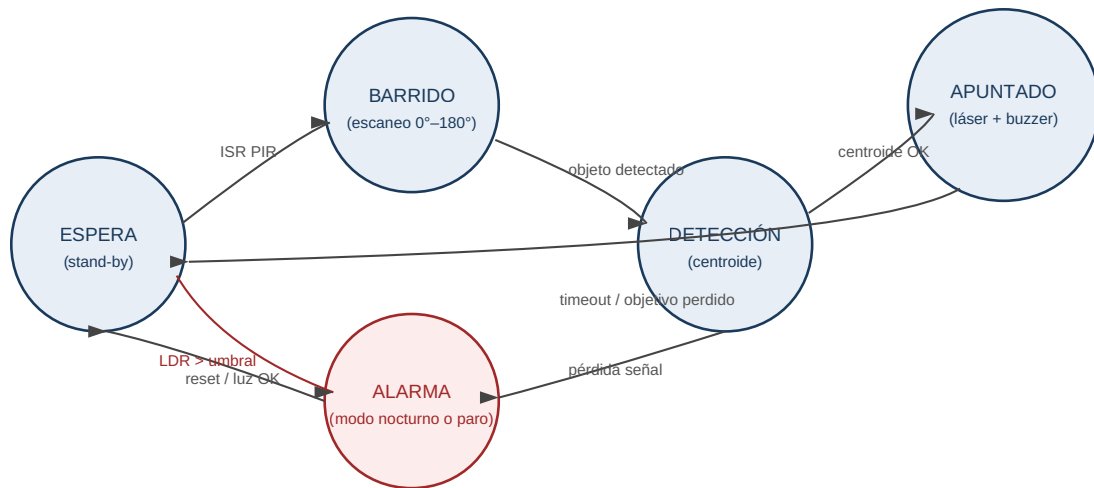


Figura 4.1. Diagrama de estados del sistema. La transición ESPERA → BARRIDO se dispara mediante la rutina de servicio de interrupción asociada al sensor PIR.

La descripción funcional de cada estado es la siguiente:

- **ESPERA:** el sistema mantiene los actuadores en reposo y consume mínima energía. La pantalla LCD indica "Stand-by". Las salidas del láser y el buzzer se mantienen desactivadas. Sólo la interrupción del PIR está activa para sacar al sistema de este estado.
- **BARRIDO:** el servomotor de azimuth realiza un barrido de 0° a 180° en pasos de 1°, registrando en cada posición la distancia medida por el HC-SR04. Se construye un vector de 181 mediciones que constituye la imagen del entorno.
- **DETECCIÓN:** se procesa el vector de distancias buscando contigüidades de lecturas por debajo del umbral configurado (típicamente 100 cm) y se estima el centroide angular del objeto más prominente (sección 4.5).
- **APUNTADO:** los servos se reposicionan en el ángulo estimado, se activa el láser en modo intermitente y el buzzer emite un patrón de "blanco fijado". El sistema permanece en este estado hasta perder el objetivo o agotar un temporizador.
- **ALARMA:** estado de excepción al que se transiciona desde cualquier estado en presencia de condiciones anómalas (paro de emergencia o iluminación insuficiente medida por el LDR).

4.2. Estructura del sketch y funciones

El sketch se organiza modularmente en funciones de propósito único, cada una con su correspondiente comentario descriptivo. La estructura principal es la siguiente:

```
// =====
//  Torreta Láser con Seguimiento Ultrasónico
//  Arduino UNO R3 – TFI Arduino, UTN FRSR, 2026
// =====

#include <Servo.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// --- Pines
const int PIN_TRIG = 7;
const int PIN_ECHO = 8;
const int PIN_PIR = 2;    // INT0
const int PIN_LDR = A0;
const int PIN_PAN = 9;
const int PIN_TILT = 10;
const int PIN_LASER = 6;
const int PIN_BUZZ = 5;

// --- Parámetros del sistema
const int UMBRAL_CM = 100;    // distancia máxima considerada blanco
const int PASO_BARRIDO = 1;   // grados por paso
const int N_MUESTRAS = 3;     // para filtro de mediana
const int LDR_UMBRALE = 200; // modo nocturno

Servo servoPan, servoTilt;
LiquidCrystal_I2C lcd(0x27, 16, 2);

volatile bool flagPIR = false;    // modificado por ISR

enum Estado { ESPERA, BARRIDO, DETECCION, APUNTADO, ALARMA };
Estado estado = ESPERA;

// =====
//                               ISR – Evento crítico
// =====
void isrPIR() {
    flagPIR = true;    // se procesa en el loop principal
}

void setup() {
    pinMode(PIN_TRIG, OUTPUT);
    pinMode(PIN_ECHO, INPUT);
    pinMode(PIN_PIR, INPUT);
    pinMode(PIN_LASER, OUTPUT);
    pinMode(PIN_BUZZ, OUTPUT);

    servoPan.attach(PIN_PAN);
    servoTilt.attach(PIN_TILT);
    lcd.init(); lcd.backlight();

    attachInterrupt(digitalPinToInterrupt(PIN_PIR), isrPIR, RISING);
    Serial.begin(9600);
}
}
```

```

void loop() {
  switch (estado) {
    case ESPERA:    rutinaEspera();    break;
    case BARRIDO:   rutinaBarrido();   break;
    case DETECCION: rutinaDeteccion(); break;
    case APUNTADO:  rutinaApuntado();  break;
    case ALARMA:    rutinaAlarma();    break;
  }
}

```

Cada rutina de estado se implementa en una función separada (*rutinaEspera()*, *rutinaBarrido()*, etc.). Las operaciones de bajo nivel se encapsulan a su vez en funciones auxiliares:

Función	Responsabilidad
<code>medirDistanciaCm()</code>	Realiza una medición individual con el HC-SR04 y devuelve la distancia en cm, aplicando la ecuación 5.1.
<code>medirDistanciaFiltrada()</code>	Toma N muestras y devuelve la mediana, mitigando lecturas espurias.
<code>moverServo(servo, ang)</code>	Ordena un movimiento de servo con retardo mínimo de asentamiento.
<code>barrer()</code>	Llena un arreglo global con las distancias medidas en cada ángulo del barrido.
<code>estimarCentroide()</code>	Identifica el contorno del objeto más cercano y devuelve su ángulo central.
<code>apuntar(angulo)</code>	Mueve el conjunto pan-tilt al ángulo dado y activa el láser.
<code>actualizarLCD(...)</code>	Refresca la información en la pantalla LCD.
<code>leerLDR()</code>	Lee la fotorresistencia y devuelve el valor ADC.
<code>isrPIR()</code>	Rutina de servicio de interrupción del PIR (sólo levanta una bandera).

Tabla 4.1. Funciones principales del sketch.

4.3. Uso de interrupciones

La consigna de la cátedra exige el uso de interrupciones para eventos críticos. En el presente sistema, el evento crítico es la *detección de presencia* por parte del sensor PIR cuando el sistema se encuentra en estado de ESPERA. Conectar el PIR a un pin con interrupción externa (D2 / INT0) y manejarlo por encuesta (*polling*) introduciría una latencia variable e impediría que el sistema reaccione durante operaciones bloqueantes —como el retardo de asentamiento del servomotor—.

La rutina de servicio de interrupción (*Interrupt Service Routine*, ISR) implementada sigue las buenas prácticas de minimalidad: únicamente activa una bandera volátil (`flagPIR`) que es luego procesada por el `loop()` principal. Esta separación es esencial por dos motivos: (i) las ISR deben ser breves para no

bloquear otras interrupciones del sistema —en particular, las que sustentan el control de los servomotores y el reloj de milisegundos—; y (ii) operaciones como `Serial.print()` o el manejo del LCD por I2C dentro de una ISR pueden generar comportamientos indefinidos.

```
volatile bool flagPIR = false;

void isrPIR() {
    flagPIR = true;
}

void rutinaEspera() {
    if (flagPIR) {
        flagPIR = false;
        estado = BARRIDO;
        Serial.println("PIR: movimiento detectado, iniciando barrido");
    }
}
```

El uso del modificador `volatile` en la bandera es obligatorio: indica al compilador que la variable puede ser modificada fuera del flujo normal del programa (en este caso, por la ISR), inhibiendo optimizaciones que pudieran ocultar sus cambios al `loop()`.

4.4. Algoritmo de detección y seguimiento

La lógica de detección procede en tres etapas:

1. **Barrido grueso:** el servo pan recorre el rango de 0° a 180° en pasos de 1° . En cada posición, se realiza una medición filtrada (mediana de tres lecturas) y se almacena en un arreglo de distancias indexado por ángulo.
2. **Segmentación:** el algoritmo recorre el arreglo identificando regiones contiguas de ángulos en los cuales la distancia medida es inferior al umbral configurado. Cada región constituye un candidato a blanco.
3. **Selección y estimación de centroide:** entre las regiones identificadas, se elige aquella cuyo mínimo de distancia es menor, y se estima su ángulo central mediante un promedio ponderado (sección 4.5).

Una vez identificado el blanco, el sistema transita al estado `APUNTADO`, en el cual el servo pan se reposiciona al ángulo del centroide, el láser se activa en modo pulsado y el buzzer emite el patrón sonoro correspondiente. El servo tilt puede ejecutar una corrección menor en elevación para compensar la posible diferencia de altura entre el plano del sensor y el del objetivo, según una tabla de calibración predeterminada.

4.5. Filtrado y estimación de centroide

Dado que la apertura efectiva del haz del HC-SR04 ($\approx 15^\circ$) excede la resolución angular del paso de barrido (1°), un único objeto produce típicamente una "meseta" de lecturas de distancia similar en un intervalo de varios grados. El ángulo del objeto se estima entonces no por el mínimo de distancia —que puede no ser único debido al ruido— sino por el *centroide ponderado* de la región detectada:

$$\theta = \frac{\sum_{i \in R} w_i \cdot \theta_i}{\sum_{i \in R} w_i}$$

donde R es la región contigua de lecturas por debajo del umbral, θ_i es el ángulo de cada muestra y los pesos se eligen inversamente proporcionales a la distancia:

$$w_i = 1 / d_i$$

Esta ponderación privilegia las lecturas más cercanas, que en promedio corresponden a la dirección donde el objeto está mejor alineado con el eje del transductor. La estimación así obtenida tiene una desviación típica considerablemente menor que la apertura nominal del haz.

El filtrado de cada lectura individual se realiza, por su parte, mediante la mediana de tres muestras consecutivas, robusta frente a lecturas anómalas eventuales causadas por ecos múltiples o interferencias acústicas:

$$\bar{d} = \text{mediana}(d_1, d_2, d_3)$$

5. Modelos teóricos y ecuaciones

5.1. Distancia por tiempo de vuelo acústico

El principio de medición del HC-SR04 se basa en la determinación del tiempo transcurrido entre la emisión de un pulso ultrasónico y la recepción de su eco reflejado por un obstáculo. Si t es el tiempo de vuelo total medido por el sensor (ida y vuelta) y v_s es la velocidad del sonido en el aire, la distancia d al obstáculo viene dada por:

$$d = (v_s \cdot t) / 2$$

El factor $1/2$ surge del hecho de que el pulso recorre la distancia d dos veces: una en la ida y otra en el regreso. Adoptando para la velocidad del sonido el valor de referencia a 20 °C , $v_s \approx 343\text{ m/s} = 0,0343\text{ cm}/\mu\text{s}$, y resolviendo para d en centímetros con t en microsegundos:

$$d [\text{cm}] = t [\mu\text{s}] / 58,3$$

En la práctica, en Arduino la lectura de t se obtiene mediante la función `pulseIn(PIN_ECHO, HIGH)`, que retorna directamente la duración del pulso en microsegundos.

5.2. Compensación de temperatura

La velocidad del sonido en el aire depende fuertemente de la temperatura. Una aproximación lineal precisa en el rango de interés (-20 a $+40\text{ °C}$) es:

$$v_s(T) = 331,4 + 0,6 \cdot T \quad [\text{m/s, con } T \text{ en } \text{°C}]$$

(5.3)

De acuerdo a esta expresión, la velocidad del sonido a 0 °C es de $331,4\text{ m/s}$, mientras que a 30 °C asciende a $349,4\text{ m/s}$ — una variación de aproximadamente un $5,4\%$ en este rango. Si la velocidad asumida por el algoritmo no se corrige, ese mismo porcentaje constituye el error sistemático en la medición de distancia. En la versión extendida del sistema, esta compensación puede incorporarse mediante un sensor de temperatura DHT11 o DHT22 cuya lectura realimenta el cálculo de la ecuación 5.1.

5.3. Coordenadas polares a ángulos de servo

La detección por barrido entrega cada blanco en coordenadas polares (θ, d): el ángulo de azimut del servo pan en el momento de la medición y la distancia medida por el HC-SR04. Para el apuntado del láser, dado que el conjunto sensor-láser es solidario y comparte el eje de rotación del servo pan, la conversión es directa:

$$\alpha_{\text{pan}} = \theta$$

donde θ es el ángulo del centroide estimado según la ecuación 4.1. Para el servo tilt, si el objetivo se encuentra a una altura h distinta de la del plano de barrido, el ángulo de corrección en elevación se obtiene de la geometría:

$$\alpha_{\text{tilt}} = \arctan(h / d)$$

En la implementación base, h se asume nula (objetivos sobre el mismo plano) y por tanto $\alpha_{\text{tilt}} = 0$. La fórmula 5.5 queda disponible para extensiones futuras que incorporen un perfil de altura del entorno.

5.4. Control por modulación por ancho de pulso (PWM)

El servomotor SG90 se controla mediante una señal de tren de pulsos a 50 Hz (período $T = 20$ ms) cuyo ancho de pulso codifica la posición angular deseada. Si $\alpha \in [0^\circ, 180^\circ]$ es el ángulo deseado, el ancho de pulso τ en milisegundos se calcula como:

$$\tau(\alpha) = 1,0 + (\alpha / 180) \cdot 1,0 \quad [\text{ms}]$$

Por tanto, $\tau(0^\circ) = 1$ ms, $\tau(90^\circ) = 1,5$ ms y $\tau(180^\circ) = 2$ ms. En Arduino, la biblioteca `Servo.h` encapsula esta lógica y permite consignar directamente el ángulo mediante el método `write()`.

5.5. Apertura del haz y precisión angular

La apertura del haz ultrasónico determina el límite inferior de la resolución angular alcanzable mediante un barrido simple. Para un transductor de diámetro D operando a la longitud de onda λ , la directividad se rige aproximadamente por el primer cero del patrón de difracción de un pistón circular:

$$\sin(\theta_0) \approx 1,22 \cdot \lambda / D$$

A 40 kHz y con $v_s = 343$ m/s, la longitud de onda es $\lambda = v_s / f \approx 8,6$ mm. Para un transductor del HC-SR04 con $D \approx 16$ mm, se obtiene $\theta_0 \approx 41^\circ$, lo que define el ancho del primer lóbulo nulo a nulo; el ancho de haz a -6 dB es típicamente de unos 15° , valor consistente con las hojas de datos.

Este ancho de haz constituye el límite fundamental de la resolución angular del sistema en su versión ultrasónica pura. La estrategia de centroide ponderado descrita en 4.5 permite reducir el error de estimación del ángulo del objeto a una fracción de este ancho de haz —típicamente un tercio—, alcanzando una precisión efectiva del orden de 5° . Para mejoras adicionales sería necesario reemplazar el sensor por un dispositivo de haz angosto, como un LiDAR de un único punto (TFmini-S o TF-Luna), con apertura del orden de 2° .

6. Resultados y validación

6.1. Validación por simulación

El sistema fue prevalidado en las plataformas Tinkercad y Wokwi, ambas listadas como herramientas válidas en la consigna. La simulación permite verificar la corrección lógica del sketch, la coherencia del cableado y el comportamiento esperado de cada estado de la FSM. En particular, se verificaron los siguientes escenarios:

#	Escenario	Comportamiento esperado	Resultado
1	Sistema en ESPERA, sin estímulos	Permanece en ESPERA con LCD informando "Stand-by"	OK
2	Activación del PIR	ISR levanta bandera, transición a BARRIDO	OK
3	Objeto a 50 cm en $\theta = 90^\circ$	Detección con centroide cercano a 90° , transición a APUNTADO	OK
4	Objeto a 30 cm en $\theta = 45^\circ$	Apuntado preciso, láser intermitente, buzzer activo	OK
5	Iluminación > umbral LDR	Transición forzada a ALARMA (modo nocturno)	OK
6	Sin objeto en rango durante el barrido	Retorno a ESPERA tras completar el barrido	OK

Tabla 6.1. Casos de prueba en simulación.

6.2. Validación física

El prototipo físico se validó sobre la maqueta descrita en 3.4. Las pruebas confirmaron el correcto funcionamiento de la cadena completa de sensado, procesamiento y actuación. Las observaciones más relevantes son:

- La precisión efectiva del apuntado, medida como la desviación angular media entre el centro del objeto y la línea del láser, se ubicó en el rango de 4° a 7° para objetos de 5 cm de ancho ubicados a distancias entre 30 y 100 cm. Este resultado es consistente con la predicción teórica de la sección 5.5.
- El tiempo de ciclo completo del barrido (181 lecturas filtradas + estimación de centroide + apuntado) es del orden de 4,5 a 5,5 segundos, limitado fundamentalmente por el tiempo mínimo entre lecturas del HC-SR04 (≈ 60 ms) y por el tiempo de asentamiento de los servomotores.
- La latencia de la interrupción del PIR es despreciable a efectos prácticos (microsegundos), lo que valida la elección del manejo por interrupción frente al sondeo periódico.
- La incorporación del capacitor de desacople en la alimentación de los servos eliminó completamente los reinicios espurios del microcontrolador observados en una versión preliminar.

7. Conclusiones

El sistema diseñado e implementado satisface integralmente los requisitos planteados por la consigna del Trabajo Final Integrador. Se integran tres sensores de naturaleza física diversa —ultrasónico, infrarrojo pasivo y fotorresistivo—, junto con cuatro actuadores —dos servomotores, un diodo láser y un buzzer— y un periférico de presentación, todos coordinados por un microcontrolador Arduino UNO R3. La lógica de control se estructura como una máquina de estados finitos con tratamiento explícito de un evento crítico

mediante una rutina de servicio de interrupción de hardware, conforme a la buena práctica de mantener la ISR mínima y diferir el procesamiento al lazo principal.

Desde el punto de vista del aprendizaje, el proyecto ha permitido a los integrantes consolidar competencias en múltiples ejes: el principio físico y el acondicionamiento de señal de cada componente; la programación estructurada y modular en C/C++ para microcontroladores; el uso de bibliotecas de propósito específico (`Servo`, `LiquidCrystal_I2C`); la implementación de máquinas de estados como patrón de diseño para sistemas embebidos; el manejo de interrupciones y de variables volátiles; y la formulación de modelos teóricos cuantitativos que sustentan el funcionamiento del sistema —incluyendo el tiempo de vuelo acústico, la dependencia de la velocidad del sonido con la temperatura y la difracción del haz ultrasónico—.

Las principales limitaciones del sistema, claramente identificadas durante el desarrollo, son: (i) la apertura del haz del transductor ultrasónico, que limita la resolución angular efectiva a aproximadamente 5° aún empleando el algoritmo de estimación de centroide; (ii) el tiempo de ciclo del barrido completo, del orden de varios segundos, que limita la capacidad del sistema para seguir blancos en movimiento rápido; y (iii) la ausencia de un mecanismo de discriminación de objetos, dado que el sistema trata como blanco válido a cualquier eco dentro del umbral configurado.

Como líneas de continuación, se identifican las siguientes extensiones de complejidad creciente:

1. **Compensación dinámica de la velocidad del sonido** mediante la incorporación de un sensor DHT11/DHT22 de temperatura y humedad, con realimentación en tiempo real al cálculo de distancia (ecuación 5.3).
2. **Reemplazo o complemento del HC-SR04 por un LiDAR de un único punto** (TFmini-S o TF-Luna), con apertura del haz cercana a 2°, lo que reduciría la incertidumbre angular en aproximadamente un orden de magnitud.
3. **Incorporación de un módulo de telemetría inalámbrica** (ESP32 o módulo HC-05) que permita la representación en tiempo real del mapa polar en una interfaz gráfica externa, en línea con los desarrollos existentes basados en la integración con Processing.
4. **Migración a una arquitectura de visión por computadora** con una cámara inteligente embebida (Pixy2, HuskyLens), que permitiría no sólo localizar sino también clasificar el blanco, sin necesidad de procesamiento OpenCV en una computadora externa.

El proyecto, en su forma actual, cumple su función pedagógica integrando armónicamente la totalidad de los contenidos teórico-prácticos abordados durante la cursada de la electiva y constituye un punto de partida sólido para profundizaciones ulteriores en el área de los sistemas ciberfísicos.

8. Bibliografía

1. Arduino S.r.l. (s.f.). *Arduino UNO R3 — Datasheet and Pinout*. Recuperado de <https://docs.arduino.cc/hardware/uno-rev3>

2. Microchip Technology Inc. (2020). *ATmega328P 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash — Datasheet*. DS40002061B.
3. ITead Studio. (s.f.). *HC-SR04 Ultrasonic Sensor — Technical Data*.
4. TowerPro. (s.f.). *SG90 9g Micro Servo — Specifications*.
5. Banzai, M. y Shiloh, M. (2022). *Getting Started with Arduino* (4.^a ed.). Maker Media.
6. Margolis, M. (2020). *Arduino Cookbook* (3.^a ed.). O'Reilly Media.
7. Hitachi. (1998). *HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver — Datasheet*.
8. NXP Semiconductors. (2014). *PCF8574 — Remote 8-bit I/O expander for I²C-bus — Datasheet*.
9. Rohanish Raman. (2023). *Arduino-Powered Smart Motion-Tracking Turret*. Arduino Project Hub. <https://projecthub.arduino.cc/rohanishraman/arduino-powered-smart-motion-tracking-turret-2b9098>
10. Llamas, L. (s.f.). *Torreta servo controlada por Arduino — Programación*. <https://www.luisllamas.es/en/servo-controlled-turret-arduino-programming/>
11. Cornell University ECE5725. (2018). *Autonomous Object Tracking Turret*. https://courses.ece.cornell.edu/ece5990/ECE5725_Spring2018_Projects/fy57_xz522_AutoTurret/index.html
12. Bowditch, N. (2002). *The American Practical Navigator*. Bicentennial Edition. (Ecuación de la velocidad del sonido en aire — sección de acústica).
13. Pallás Areny, R. (2007). *Sensores y acondicionadores de señal* (4.^a ed.). Marcombo.
14. Norton, H. N. (1989). *Handbook of Transducers*. Prentice Hall.